

Unit 5

Strings and Characters



INTRODUCTION

- String represents a group of characters.
- Strings are important because most of the data that we use in daily life will be in the form of strings. For example, the names of persons, their addresses, vehicle numbers, their credit card numbers, etc. are all strings.
- In Python, the str datatype represents a string, and **str** is *built in class* in python
- Since every string comprises several characters, Python handles strings and characters almost in the same manner. There is no separate datatype to represent individual characters in Python



CREATING STRINGS

- We can create a string in Python by assigning a group of characters to a variable. The group of characters should be enclosed inside single quotes or double quotes as:

```
s1 = 'Welcome to Core Python learning'  
s2 = "welcome to Core Python learning"
```

- Sometimes, we can use triple single quotes or triple double quotes to represent strings. These quotation marks are useful when we want to represent a string that occupies several lines as:

```
str1 = '''welcome to Core Python, a book on Python language that discusses all important concepts of  
Python in a lucid and comprehensive manner.'''
```

- It is possible to display quotation marks to mark a sub string in a string

```
s1 = 'welcome to "Core Python" learning'  
print(s1)
```



- It is possible to use escape characters like \t or \n inside the strings.

```
sl = "Welcome to \t Core Python \n learning"
```

```
print(sl)
```

- Escape Characters and their Meanings is in the following table

Escape Character	Meaning
\a	Bell or alert
\b	Backspace
\n	New line
\t	Horizontal tab space
\v	Vertical tab space
\r	Enter button
\x	Character x
\\	Displays single \



- To nullify the effect of escape characters, we can create the string as a 'raw' string by adding 'r' before the string as:

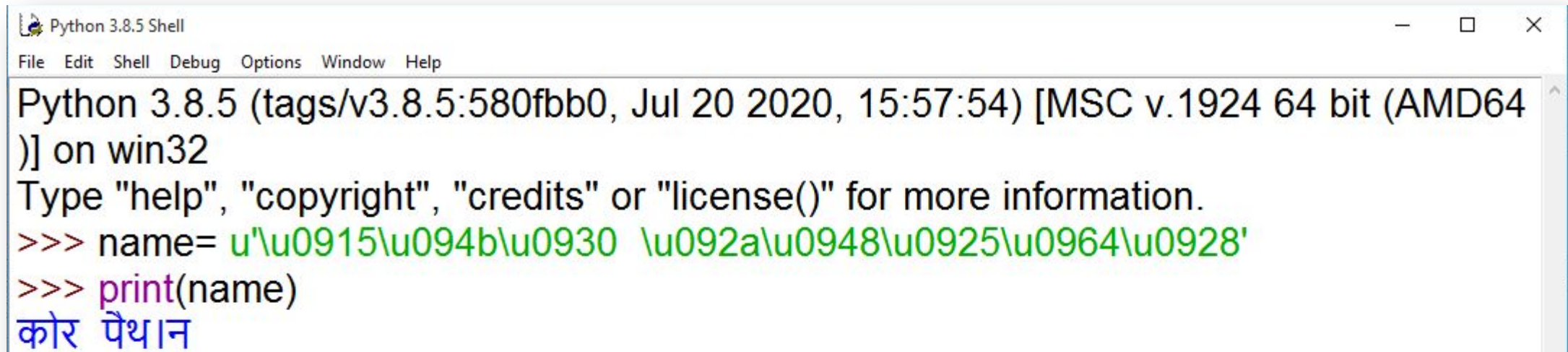
```
s1= r"welcome to\tCore Python\nlearning"  
print(s1)
```

- This is not showing the effect of \t or \n
- The preceding lines of code will display the following output:

```
welcome to\tCore Python\ nlearning
```



- To create a string with Unicode characters, we should add 'u' at the beginning of the string
- For example, it is possible to display the alphabet of Hindi, French, and German languages using Unicode system. Each Unicode character contains 4 digits preceded by a \u. The following statement displays 'Core Python' in Hindi using Unicode characters. There are 8 Unicode characters used for this purpose.

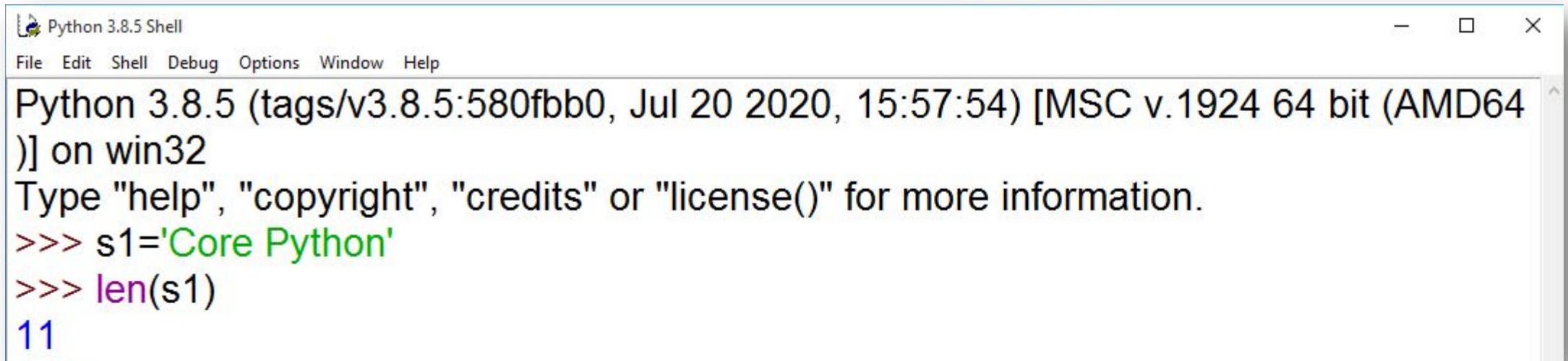


```
Python 3.8.5 Shell
File Edit Shell Debug Options Window Help
Python 3.8.5 (tags/v3.8.5:580fbb0, Jul 20 2020, 15:57:54) [MSC v.1924 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> name= u"\u0915\u094b\u0930 \u092a\u0948\u0925\u0964\u0928"
>>> print(name)
कोर पैथान
```



LENGTH OF A STRING

- Length of a string represents the number of characters in a string.
- To know the length of a string, we can use the `len()` function.
- This function gives the number of characters including spaces in the string.



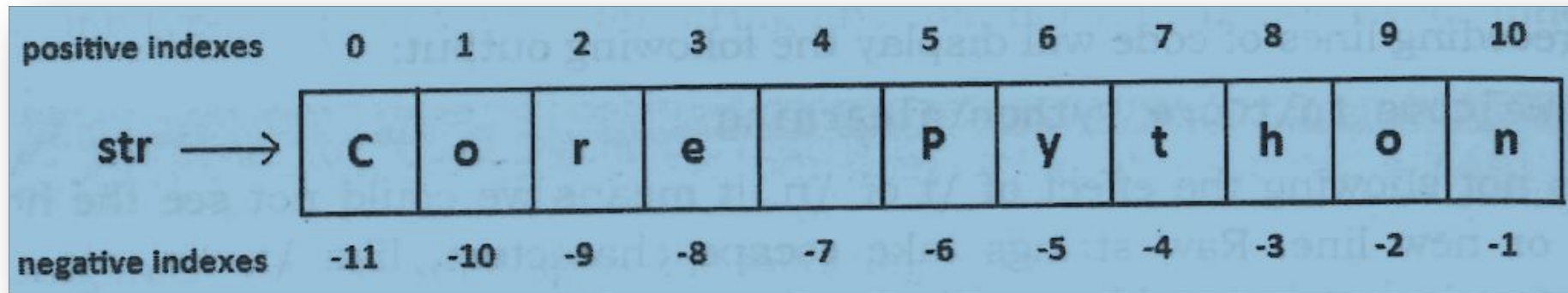
```
Python 3.8.5 Shell
File Edit Shell Debug Options Window Help
Python 3.8.5 (tags/v3.8.5:580fbb0, Jul 20 2020, 15:57:54) [MSC v.1924 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> s1='Core Python'
>>> len(s1)
11
```



INDEXING IN STRINGS

- Index represents the position number. Index is written using square braces [].
- By specifying the position number through an index, we can refer to the individual elements (or characters) of a string.
- For example, `str[0]` refers to the 0th element of the string and `str[1]` refers to the 1st element of the string.
- Thus, `str[i]` can be used to refer to i^{th} element of the string. Here, 'i' is called the string index because it is specifying the position number of the element in the string.
- When we use index as a negative number, it refers to elements in the reverse order.
- `str[-1]` refers to the last element and `str[-2]` refers to second element from last and so on to the first element





- [Go to Jupyter notebook for examples](#)



SLICING THE STRINGS

- A slice represents a part or piece of a string.
- The format of slicing is:
stringname[start: stop: stepsize]
- If 'start' and 'stop' are not specified, then slicing is done from 0th to n-1th index, if 'stepsize' is not written, then it is taken to be 1
- [Go to Jupyter notebook for examples](#)



REPEATING THE STRINGS

- The repetition operator is denoted by " symbol and is useful to repeat the string for several times. For example, `str * n` repeats the string for `n` times. See the example:

```
str = 'Core Python'  
print(str*2)
```

- The preceding lines of code will display the following output:

```
Core PythonCore Python
```

- Similarly, it is possible to repeat a part of the string obtained by slicing as:

```
s = str[5:7]*3    # repeat 5th 6th characters for 3 times  
print(s)
```

- The output of the preceding statement is as follows:

```
Py Py Py
```



CONCATENATION OF STRINGS

- We can use '+' on strings to attach a string at the end of another string.
- This operator '+' is called addition operator when used on numbers.
- But, when used on strings, it is called 'concatenation' operator since it joins or concatenates the strings.

```
>>> s1='Core'
```

```
>>> s2='Python'
```

```
>>> s1+s2
```

```
'CorePython'
```



CHECKING MEMBERSHIP

- We can check if a string or character is a member of another string or not using 'in' or 'not in' operators.
- The 'in' operator returns True if the string or character is found in the main string. It returns False if the string or character is not found in the main string.
- The 'not in' operator returns False if a string or character is not found in the main string, otherwise True.
- The operators 'in' and 'not in' make **case sensitive comparisons**. It means these operators consider the upper case and lower case letters or strings differently while comparing the strings.
- [Go to Jupyter notebook for examples](#)



COMPARING STRINGS

- We can use the relational operators like `>`, `>=`, `<`, `<=`, `==` or `!=` operators to compare two strings.
- They return Boolean value, i.e. either True or False depending on the strings being compared
- While comparing the strings, python interpreter compares them by taking them in English dictionary order. The string which comes first in the dictionary order will have a low value than the string which comes next.
- It means, 'A' is less than 'B' which is less than 'C' and so on
- [Go to Jupyter notebook for examples](#)



REMOVING SPACES FROM A STRING

- A space is also considered as a character inside a string.
- Sometimes, the unnecessary spaces in a string will lead to wrong results. For example

```
if ' Mukesh ' == 'Mukesh':  
    print('Welcome')  
else:  
    print('name not found')
```

Output:
name not found
- Hence such spaces should be removed from the strings before they are compared.
- This is possible using `rstrip()`, `lstrip()` and `strip()` methods.
- The `rstrip()` method removes the spaces which are at the right side of the string. The `lstrip()` method removes spaces which are at the left side of the string. `strip()` method removes spaces from both the sides of the strings
- [Go to Jupyter notebook for examples](#)



FINDING SUB STRINGS

- The find(), rfind(), index() and rindex() methods are useful to locate sub strings in a string.
- These methods return **the location of the first occurrence of the sub string** in the main string.
- The find() and index() methods search for the sub string from the beginning of the main string.
- The rfind() and rindex() methods search for the sub string from right to left, i.e. in backward order.
- The find() method returns -1 if the sub string is not found in the main string.
- The index() method returns 'ValueError' exception if the sub string is not found.
- The format of find() method is:
mainstring.find(substring, beginning, ending)
- The same format is used for other methods also.
- [Go to Jupyter notebook for examples](#)



COUNTING SUBSTRINGS IN A STRING

- The method `count()` is available to count the number of occurrences of a sub string in a main string.
- The format of this method is:

`stringname.count (substring)`

- This returns an integer number that represents how many times the substring is found in the main string.
- We can limit our search by specifying beginning and ending positions in the `count()` method so that the substring position is counted only in that range. Hence, the other form of `count()` method is:

`stringname.count(substring,begin, end)`

- [Go to Jupyter notebook for examples](#)



STRINGS ARE IMMUTABLE

- An *immutable* object is an object whose content cannot be changed. On the other hand, a *mutable* object is an object whose content can be changed as and when required.
- In Python, *numbers, strings and tuples* are *immutable*.
- *Lists, sets, dictionaries* are *mutable* objects.
- There are two reasons why string objects are made immutable in Python.
 - **Performance:** When an object is immutable, it will have fixed size in memory since it cannot be modified. Because strings are immutable, we can easily allocate memory space for them at creation time, and the storage requirements are fixed and unchanging. Hence it takes less time to allocate memory for strings and less time to access them. This increases the performance of the software.
 - **Security:** Since string objects are immutable, any attempts to modify the existing string object will create a new object in memory. Thus the identity number of the new object will change that lets the programmer to understand that somebody modified the original string

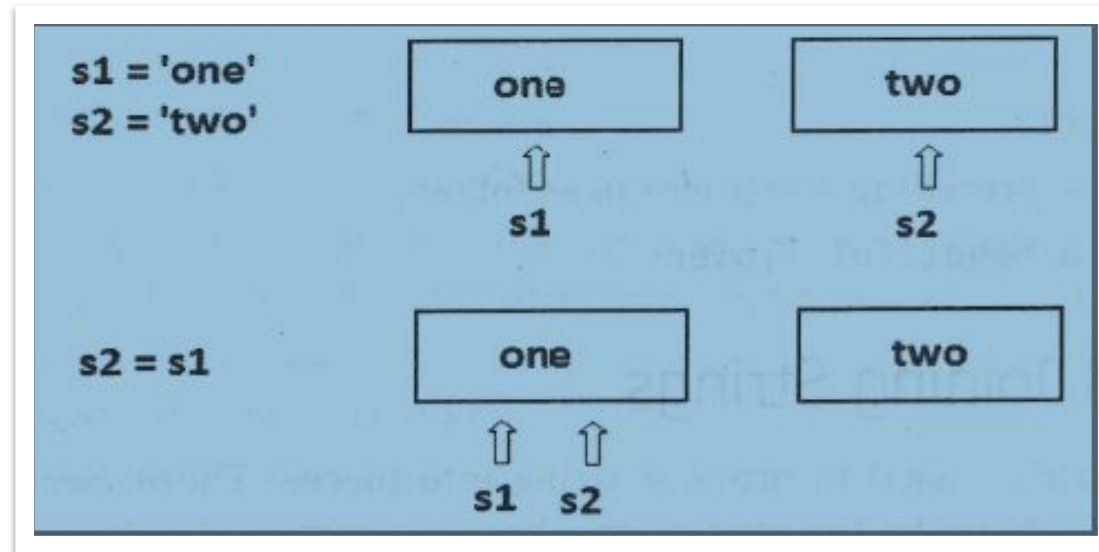


- Examples:

```
>>> s='abcd'
>>> print(s[0])
a
>>> s[0]='x'
Traceback (most recent call last):
  File "<pyshell#2>", line 1, in <module>
    s[0]='x'
TypeError: 'str' object does not support item assignment
```



```
>>> s1='one'
>>> s2='two'
>>> id(s1)
533804836656
>>> id(s2)
533804836592
>>> s2=s1
>>> id(s1)
533804836656
>>> id(s2)
533804836656
```



REPLACING A STRING WITH ANOTHER STRING

- The `replace()` method is useful to replace a sub string in a string with another sub string. The format of using this method is:

stringname.replace(old, new)

- This will replace all the occurrences of 'old' sub string with 'new' sub string in the main string.

- Exmample:

```
>>> s='It is a beautiful horse'
```

```
>>> s1='horse'
```

```
>>> s2='house'
```

```
>>> newstr=s.replace(s1,s2)
```

```
>>> print(s)
```

```
It is a beautiful horse
```

```
>>> print(newstr)
```

```
It is a beautiful house
```



SPLITTING AND JOINING STRINGS

- The `split()` method is used to brake a string into pieces.
- These pieces are returned as a list.
- By default split method takes space as a separator
- For example, to brake the string `str` where a comma (,) is found, we can write:
`str.split(',')`
- When a group of strings are given, it is possible to join them all and make a single string. For this purpose, we can use `join()` method as follows
`separator.join(str)`
Where the 'separator' represents the character to be used between the strings
- [Go to Jupyter notebook for examples](#)



CHANGING CASE OF STRING

- Python offers 4 methods that are useful to change the case of a string.
- They are `upper()`, `lower()`, `swapcase()`, `title()`.
- The `upper()` method is used to convert all the characters of a string into uppercase or capital letters.
- The `lower()` method converts the string into lowercase or into small letters.
- The `swapcase()` method converts the capital letters into small letters and vice versa.
- The `title()` method converts the string such that each word in the string will start with a capital letter and remaining will be small letters.
- [Go to Jupyter notebook for examples](#)



CHECKING STARTING AND ENDING OF A STRING

- The startswith() method is useful to know whether a string is starting with a sub string or not. The way to use this method is:

str.startswith(substring)

- When the sub string is found in the main string 'str', this method returns True. If the sub string is not found, it returns False.
- Similarly, to check the ending of a string, we can use endwith() method. It returns True if the string ends with the specified sub string, otherwise it returns False
- [Go to Jupyter notebook for examples](#)



STRING TESTING METHODS

- There are several methods to test the nature of characters in a string.
- These methods return either True or False.
- For example, if a string has only numeric digits, then `isdigit()` method returns True.
- These methods can also be applied to individual characters
- In the following table mentioned the string and character testing methods:

Method	Description
<code>isalnum()</code>	This method returns True if all characters in the string are alphanumeric (A to Z, a to z, 0 to 9) and there is at least one character; otherwise it returns False.



Method	Description
isalpha()	Returns True if the string has at least one character and all characters are alphabetic (A to Z and a to z); otherwise, it returns False.
isdigit()	Returns True if the string contains only numeric digits (0 to 9) and False otherwise.
islower()	Returns True if the string contains at least one letter and all characters are in lower case; otherwise, it returns False.
isupper()	Returns True if the string contains at least one letter and all characters are in upper case; otherwise, it returns False.
istitle()	Returns True if each word of the string starts with a capital letter and there is at least one character in the string; otherwise, it returns False.
isspace()	Returns True if the string contains only spaces; otherwise, it returns False.

- [Go to Jupyter notebook for examples](#)



SORTING STRINGS

- We can sort a group of strings into alphabetical order using `sort()` method and `sorted()` function.
- The `sort()` method is used in the following way:

```
str.sort()
```

Here, ``str`` represents an array that contains a group of strings.

- When `sort()` method is used, it will sort the original array, i.e. ``str``. So, the original order of strings will be lost and we will have only one sorted array.
- To retain the original array even after sorting, We can use `sorted()` function as:

```
str1 = sorted(str)
```

Here, ``str`` is the original array whose elements are to be sorted. After sorting the array, the sorted array will be referenced by ``str1``. So, the sorted strings appear in the array ``str1``. The original array ``str`` is undisturbed

[Go to Jupyter notebook for examples](#)



SEARCHING IN THE STRING

- The easiest way to search for a string in a group of 'n' strings is by using sequential search or linear search technique.
- This is done by comparing the searching string 's' with every string in the group.
- For this purpose, we can use a for loop that iterates from 0th to n-1th string in the group. By comparing the searching string 's' with every one of these strings, we can decide whether 's' is available in the group or not
- [Go to Jupyter notebook for examples](#)

